

Special Participation B: Coding Problem in HW 6 using Deepseek

Muon Optimizer

Yicheng Xiao

3042011144

jaredanjerry@berkeley.edu

? Question:

In this problem, we will implement key components of the Muon optimizer in PyTorch. Then, we will compare Muon against SGD and AdamW empirically to demonstrate the benefits of Muon.

The core of Muon is using matrix orthogonalization on a typical gradient update matrix. After orthogonalization:

- Singular values become more uniform
- Updates act across all directions in parameter space
- The neural network can utilize its full parameter capacity (because parameters are all receiving nontrivial gradients)

In Muon, matrix orthogonalization is done using Newton-Schulz. Newton-Schulz relies on iteration using an odd matrix polynomial. In this problem, we consider a very simple cubic polynomial, but many odd polynomials can be used (state-of-the-art implementations use specific, tuned quintic polynomial).

Complete the following code to implement Newton-Schulz, using the matrix polynomial in the comments.

```
def newton_schulz_orthogonalize(X: torch.Tensor, num_iters: int):
    """
    Apply Newton-Schulz iterations to approximate orthogonalization.

    This function applies the polynomial  $f(X) = (3X - X^3)/2$  repeatedly to a normalized matrix,
    which gradually forces all singular values to 1 while preserving singular vectors.

    Args:
        X (torch.Tensor): Input matrix to orthogonalize
        num_iters (int): Number of Newton-Schulz iterations

    Returns:
        torch.Tensor: Orthogonalized matrix
    """
    dtype = X.dtype
    # Use bfloat16 for potential speed/memory savings during the iterations
    X = X.bfloat16()
    # Recall from prior homeworks that we can transpose the matrix to speed up computation.
    transposed = False
    if X.size(-2) < X.size(-1):
        transposed = True
        X = X.mT

    # Ensure spectral norm is at most sqrt(3)
    norm = torch.linalg.norm(X, dim=(-2, -1), keepdim=True)
    X = torch.div(X, norm + 1e-7) * (3**0.5)

    #####
    # TODO: YOUR CODE HERE
    #####

    if transposed:
        X = X.mT

    return X.to(dtype)
```

Think for 10 seconds

To solve this problem, we need to implement the Newton-Schulz iteration method for matrix orthogonalization. This method uses a cubic polynomial to iteratively adjust the singular values of a matrix towards 1, effectively orthogonalizing it. The key steps involve normalizing the matrix, applying the polynomial transformation iteratively, and handling transposition for efficiency.

Approach

1. **Transpose Handling:** If the number of rows is less than the number of columns, transpose the matrix to optimize computation. This ensures that the larger dimension is handled efficiently during matrix operations.
2. **Normalization:** Normalize the matrix by its spectral norm and scale it by $\sqrt{3}$ to ensure the spectral norm is at most $\sqrt{3}$, which is necessary for the Newton-Schulz iteration to converge.
3. **Iterative Application:** Apply the Newton-Schulz polynomial transformation iteratively. The polynomial used is $f(X) = \frac{3X - X^3}{2}$, which gradually adjusts the singular values towards 1.
4. **Transpose Reversal:** After completing the iterations, revert the transposition if initially applied to return the matrix to its original orientation.

Solution Code

```
def newton_schulz_orthogonalize(X: torch.Tensor, num_iters: int):
    """
    Apply Newton-Schulz iterations to approximate orthogonalization.

    This function applies the polynomial  $f(X) = (3X - X^3)/2$  repeatedly to a normalized matrix,
    which gradually forces all singular values to 1 while preserving singular vectors.

    Args:
        X (torch.Tensor): Input matrix to orthogonalize
        num_iters (int): Number of Newton-Schulz iterations

    Returns:
        torch.Tensor: Orthogonalized matrix
    """
    dtype = X.dtype
    # Use bfloat16 for potential speed/memory savings during the iterations
    X = X.bfloat16()
    # Recall from prior homeworks that we can transpose the matrix to speed up computation.
    transposed = False
    if X.size(-2) < X.size(-1):
        transposed = True
        X = X.mT

    # Ensure spectral norm is at most sqrt(3)
    norm = torch.linalg.norm(X, dim=(-2, -1), keepdim=True)
    X = torch.div(X, norm + 1e-7) * (3**0.5)

    #####
    # TODO: YOUR CODE HERE
    #####
    for _ in range(num_iters):
        X = (3 * X - X @ X.mT @ X) / 2
    #####

    if transposed:
        X = X.mT

    return X.to(dtype)
```

Explanation

1. **Transpose Handling:** The matrix is transposed if the number of rows is less than the number of columns to optimize matrix multiplication performance.
2. **Normalization:** The matrix is normalized by its spectral norm and scaled by $\sqrt{3}$ to ensure the spectral norm condition required for convergence.
3. **Iterative Application:** The Newton-Schulz polynomial is applied iteratively. Each iteration refines the matrix towards orthogonality by adjusting its singular values.

4. **Transpose Reversal:** After iterations, the matrix is transposed back to its original form if necessary, ensuring the output maintains the input's dimensional orientation.

This approach efficiently orthogonalizes the matrix using iterative polynomial transformations, leveraging matrix properties and normalization to ensure convergence and correctness.

★ **Good Point!**

Correct at one-shot!

? **Question:**

Notice that in the above implementation, we scale the spectral norm to be at most $\sqrt{3}$. Can you explain why we choose this particular scaling?

! **Oops, something might be wrong...**

Thinking time exceeds the limits.

? **Question:**

Notice that in the above implementation, we scale the spectral norm to be at most $\sqrt{3}$. Can you explain why we choose this particular scaling? Just explain is enough, you don't have to provide codes for this.

Think for 29 seconds

The scaling by $\sqrt{3}$ is chosen for mathematical reasons related to the convergence properties of the Newton-Schulz iteration. Here's the explanation:

Convergence Region Analysis

The Newton-Schulz iteration uses the polynomial:

$$f(X) = (3X - X^3)/2$$

For this iteration to converge to an orthogonal matrix, the initial matrix must satisfy certain spectral conditions:

1. **Singular Value Mapping:** If we consider the effect on singular values σ , the iteration becomes:

$$\sigma \mapsto (3\sigma - \sigma^3)/2$$

2. **Fixed Points:** The fixed points of this mapping occur when:

$$\sigma = (3\sigma - \sigma^3)/2$$

Solving this gives $\sigma = 0$, $\sigma = 1$, and $\sigma = -1$.

3. **Convergence Region:** For convergence to $\sigma = 1$, we need the mapping to be a contraction in the neighborhood of 1. Analysis shows that the iteration converges to orthogonal matrices when all singular values satisfy $|\sigma| < \sqrt{3}$.

4. **Why $\sqrt{3}$ specifically:**

- The derivative of $f(\sigma) = \frac{3\sigma - \sigma^3}{2}$ is $f'(\sigma) = \frac{3 - 3\sigma^2}{2}$
- For the iteration to be stable and convergent, we need $|f'(\sigma)| < 1$ in the region of interest
- The critical point where $|f'(\sigma)| = 1$ occurs at $\sigma = \sqrt{1 \pm \frac{2}{3}}$, giving the boundary around $\sigma = \sqrt{3}$
- If any singular value exceeds $\sqrt{3}$, the iteration can diverge

5. **Optimal Scaling:** By scaling the matrix so its spectral norm is exactly $\sqrt{3}$, we:

- Ensure all singular values are $\leq \sqrt{3}$ (satisfying the convergence condition)
- Maximize the convergence rate by starting at the boundary of the stable region
- Avoid the trivial fixed point at $\sigma = 0$ while ensuring convergence to $\sigma = 1$

This scaling ensures the Newton-Schulz iteration will reliably converge to an orthogonal matrix while preserving the singular vectors of the original matrix.

★ **Good Point!**

This is generally correct!

? Question:

Now, we implement the update in a Muon optimizer. Given parameter matrix W with momentum matrix M , the pseudocode for the Muon update proceeds as follows:

```
d_out, d_in = M.shape
```

```
# Apply Newton-Schulz orthogonalization
M ← newton_schulz_orthogonalize(M, ns_iters)
```

```
# Apply muP scaling factor for consistent update magnitude
M ← M · sqrt(max(1, d_out / d_in))
```

Then, the Muon update is used later to update the parameters W :

```
# Update the parameter matrix
W ← W - lr * M
```

Complete the following code to implement the Muon update following the above pseudocode.

```
import torch.optim as optim

def muon_update(grad, momentum, beta=0.95, ns_iters=5):
    momentum.lerp_(grad, 1 - beta) # momentum = beta * momentum + (1-beta) * grad
    update = momentum.clone()

    # If the parameter is a convolutional kernel, then flatten to a 2D matrix
    original_shape = update.shape
    reshaped = False
    if update.ndim > 2:
        reshaped = True
        update = update.view(update.size(0), -1)

    #####
    # TODO: YOUR CODE HERE
    #####

    # Restore shape if needed
    if reshaped:
        update = update.view(original_shape)

    return update
```

Think for 10 seconds

Looking at the pseudocode and the provided code structure, I need to implement the Muon update by:

1. Applying Newton-Schulz orthogonalization to the momentum matrix
2. Applying the muP scaling factor based on the matrix dimensions
3. Returning the processed update

Here's the implementation:

```
import torch.optim as optim

def muon_update(grad, momentum, beta=0.95, ns_iters=5):
    momentum.lerp_(grad, 1 - beta) # momentum = beta * momentum + (1-beta) * grad
    update = momentum.clone()

    # If the parameter is a convolutional kernel, then flatten to a 2D matrix
    original_shape = update.shape
    reshaped = False
    if update.ndim > 2:
        reshaped = True
        update = update.view(update.size(0), -1)
```

```
#####
# TODO: YOUR CODE HERE
#####
# Apply Newton-Schulz orthogonalization
update = newton_schulz_orthogonalize(update, ns_iters)

# Apply muP scaling factor for consistent update magnitude
d_out, d_in = update.shape
scale_factor = torch.sqrt(torch.max(torch.tensor(1.0), torch.tensor(d_out / d_in)))
update = update * scale_factor
#####

# Restore shape if needed
if reshaped:
    update = update.view(original_shape)

return update
```

Explanation:

1. **Newton-Schulz Orthogonalization:** After flattening the update matrix if needed, we apply the `newton_schulz_orthogonalize` function that was implemented earlier. This transforms the momentum matrix to have more uniform singular values.
2. **muP Scaling:** Following the pseudocode, we:
 - Extract the dimensions `d_out` (number of rows) and `d_in` (number of columns)
 - Calculate the scaling factor as `sqrt(max(1, d_out / d_in))`
 - Scale the orthogonalized matrix by this factor
3. **Shape Restoration:** If we flattened a convolutional kernel, we restore it to its original shape before returning.

The muP scaling (`max(1, d_out/d_in)`) is important for maintaining consistent update magnitudes across layers with different dimensionalities, which helps with training stability in the muP (maximal update parameterization) framework that Muon is built upon.

This implementation follows the exact pseudocode provided, applying both the orthogonalization and scaling steps before returning the processed update that will later be used to update the parameters with $W \leftarrow W - \eta r * M$.

★ Good Point!

This is correct! Just not in the syntax that we expect. We can use `update.mul_(max(1, update.size(-2) / update.size(-1))**0.5)` to simplify this.

? Question:

Note that Muon requires that parameters are 2D matrices of shape $d_{\text{out}} \times d_{\text{in}}$. However, we know that parameters that are convolutional kernels have shape $c_{\text{out}} \times c_{\text{in}} \times k \times k$ where c denotes number of channels and k is kernel size.

Modern implementations of convolutional layers will transform an input image x of shape $c_{\text{in}} \times h \times w$ to x' such that each column has size $c_{\text{in}} \cdot k \cdot k$ and corresponds to one flattened “receptive field” of the image (or one patch of the image that a convolutional filter passes over to compute one pixel in the output).

Given this fact, **how do we modify the convolutional kernel into a $d_{\text{out}} \times d_{\text{in}}$ matrix C such that the output of the convolutional layer can be expressed as Cx' .**

Think for 30 seconds

Based on the transformation of the input image x into x' where each column represents a flattened receptive field of size $c_{\text{in}} \cdot k \cdot k$, the convolutional kernel should be reshaped as follows:

Reshaping the Convolutional Kernel

The convolutional kernel with shape $c_{\text{out}} \times c_{\text{in}} \times k \times k$ should be reshaped into a 2D matrix C of dimensions:

$$d_{\text{out}} = c_{\text{out}} \quad d_{\text{in}} = c_{\text{in}} \cdot k \cdot k$$

Mathematical Justification

- 1. **Original convolution operation:** Each output channel is computed by applying a 3D filter (size $c_{in} \times k \times k$) across the spatial dimensions of the input.
- 2. **Matrix formulation:** When we transform the input x into x' where:
 - Each column of x' has dimension $c_{in} \times k \times k$ (flattened receptive field)
 - The number of columns equals the number of output spatial positions ($h_{out} \times w_{out}$)
- 3. **Matrix multiplication:** The convolution can then be expressed as:

$$y' = C \cdot x'$$

where:

- C has shape $c_{out} \times (c_{in} \times k \times k)$
 - x' has shape $(c_{in} \times k \times k) \times (h_{out} \times w_{out})$
 - y' has shape $c_{out} \times (h_{out} \times w_{out})$
4. **Reshaping back:** The output y' can then be reshaped to the final output dimensions $c_{out} \times h_{out} \times w_{out}$.

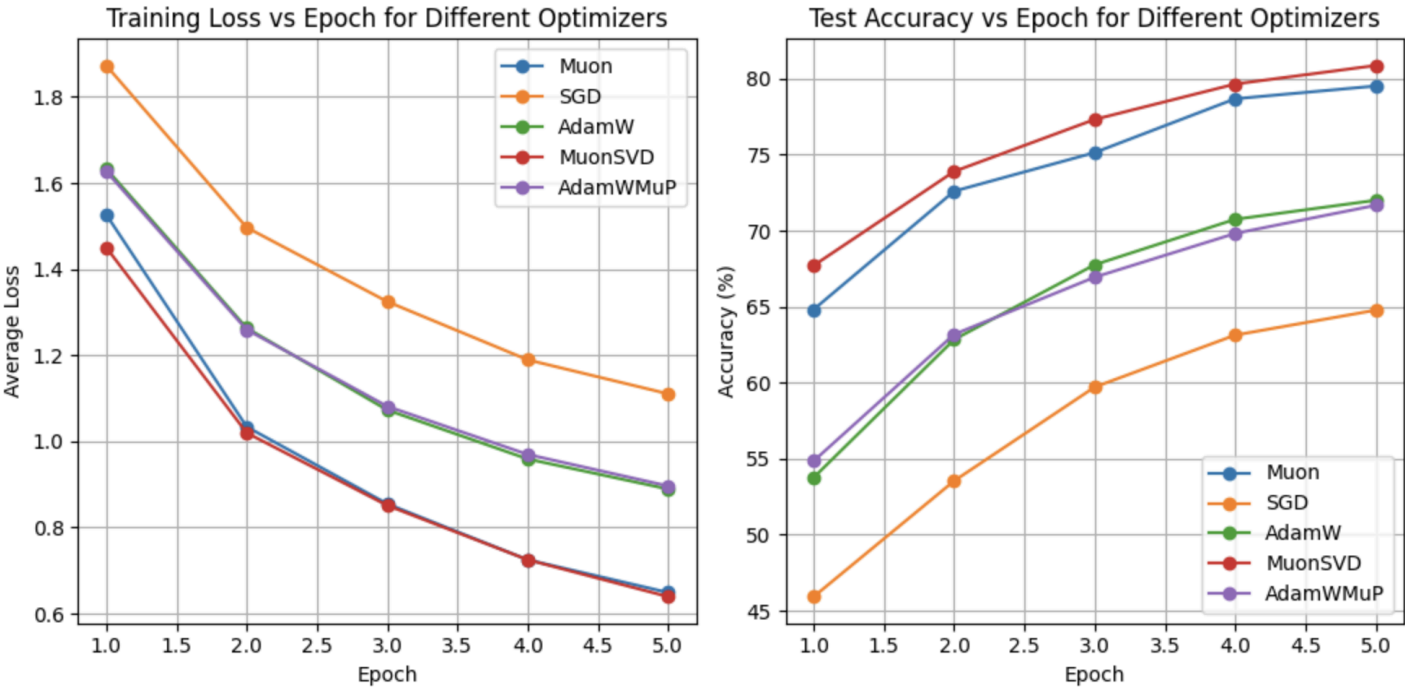
Implementation in Muon

This reshaping is exactly what the provided code does:

```
if update.ndim > 2:
    reshaped = True
    update = update.view(update.size(0), -1) # Shape becomes (c_out, c_in * k * k)
```

This transformation allows Muon to treat convolutional kernels as 2D matrices for orthogonalization while preserving the mathematical equivalence of the convolution operation when expressed as matrix multiplication Cx' .

★Good Point!
This is also correct!



Because the model is lack of multimodal ability, Deepseek cannot read the final result and solve the following problems:

? Question:

1. Which optimizer performed best between Muon, SGD, and AdamW? Also copy the resulting plots into the submission as well.
2. Compare the loss curves between Muon and MuonSVD. Are the results expected? Explain why.
3. The Muon optimizer contains two key differences: (1) orthogonalization of the momentum, and (2) muP scaling of the momentum. Between orthogonalization and muP scaling, which seemed to matter more? Reference the loss curves to justify your answer.